

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey



Diseño de sistemas en chips
TE2003B, Grupo 500

Enrique González Guerrero

Reto: Sistema de infoentretenimiento

Equipo #10 | Integrantes:

Marcelo Ruiz | A01197644

Octavio Martínez | A01383532

Luis Molina | A00829530

Eduardo de la Rosa | A01039969

Junio 2022

Introducción

Como parte del curso de Diseño de Sistemas en Chip, se ha desarrollado un prototipo de un sistema de infoentretenimiento, en donde un usuario del sistema podrá seleccionar una de las canciones que vienen integradas en este mismo, además de que podrá ajustar su volumen, usar “forward”, y “backward”. La intención de este proyecto es demostrar la capacidad de modelación de un sistema embebido, el uso correcto de una unidad de procesamiento y la configuración e integración de un protocolo de comunicación entre los dispositivos que conforman a dicho sistema de infoentretenimiento.

Los dispositivos principales que conforman el sistema son los siguientes:

- Raspberry Pi
- STM32

Justificación del problema

El prototipo se basa en un sistema de infoentretenimiento como el que se puede encontrar en la mayoría de los autos modernos. Vamos a utilizar el microcontrolador como un usuario, es decir, recibe la información de lo que el usuario quiere hacer, y el Raspberry Pi como un servidor, en donde se reproducirá la música seleccionada por el usuario y podrá manipularla con diferentes funciones.

Metodología

La Raspberry Pi tiene la capacidad de reproducir música mediante el microcontrolador STM32. El sistema para almacenar las canciones es una tarjeta SD (Secure Digital). El sistema tiene un LCD para mostrar datos de la canción que se está reproduciendo tales como: nombre de la canción y artista.

Se utilizan los botones con carácter especial para lo siguiente:

- “Play/Pause”
 - Botón utilizado para dar pausa y continuar la reproducción de una canción.
- “Rewind”
 - Botón que se regresa al inicio de una canción.
- “Forward”
 - Botón que reproduce la siguiente canción

Manual de instrucciones

Para el uso correcto de prototipo construido existen los siguientes comandos que fueron programados e integrados en el código python:



- ☐ **0-9:** Selección de canciones
- ☐ **A:** Pausa
- ☐ **B:** Detener programa
- ☐ **C:** Siguiente
- ☐ **D:** Anterior
- ☐ ***:** Bajar volumen
- ☐ **#:** Subir volumen

A continuación se mostrará la lista de las canciones que fueron utilizadas para demostrar el funcionamiento de los números en el teclado matricial.

0. *Just Like Heaven* - The Cure
1. *La Dama y el Moribundo* - Jose Madero
2. *This Charming Man* - The Smiths
3. *All Those Friendly People* - Funeral Suits
4. *Time To Pretend* - MGMT
5. *Everlong* - Foo Fighters
6. *Eye of the Tiger* - Survivor
7. *Another One Bites the Dust* - Queen
8. *American Woman* - The Guess Who
9. *Rocketman* - Elton John

Se deberá de presionar el botón correspondiente a la canción una sola vez para que esta después se reproduzca. Mantener el botón de alguna canción presionado no reproducirá la canción hasta que este sea soltado.

Se puede mantener presionado los botones de control de volumen para facilitar su uso.

Marco teórico

- Linux OS

Linux es un sistema operativo que cuenta con bastantes librerías, facilitando su uso por ofrecer muchas funcionalidades que se integran rápidamente. Este sistema operativo es compatible con Bash, diseñado para su uso en Linux.

- Raspberry Pi (RPi)

La Raspberry Pi es una placa de desarrollo de la serie PI y puede considerarse como una computadora que trabaja con el sistema operativo de Linux. La placa tiene una buena rapidez de procesamiento para correr aplicaciones, especialmente para sistemas embebidos y sistemas de IoT. La RPi tiene pines de fuente de alimentación de:

- +5V
- +3.3V
- GND
- Vin input

También podemos encontrar pines de comunicación:

- Asíncrona
 - UART
 - GPIO
 - 15
 - 14
- Sincrónica
 - MOSI
 - MISO
 - CLK
 - GPIO
 - SPI0
 - 10
 - 9
 - 11
 - 8
- SPI1 (Serial Peripheral Interface)
 - GPIO
 - 20
 - 19
 - 21
 - 7

Pines de INPUT OUTPUT, PWM, y External interrupts(All I/O).

Además la RPi tiene la posibilidad de conectar otros dispositivos/periféricos por medio de USB, HDMI's y Jack.

Para descargar un sistema operativo se puede hacer por medio de una tarjeta MicroSD que se puede introducir a la RPi.

- **STM32F103x6**

El microcontrolador que se usará es una STM32. La familia de línea de rendimiento STM32F103x6 incorpora lo siguiente:

- Núcleo RISC de 32 bits ARM® Cortex-M3 de alto rendimiento
 - funciona a una frecuencia de 72 MHz
- Memorias integradas de alta velocidad
 - Memoria Flash de hasta 32 Kbytes
 - SRAM de hasta 6 Kbytes
- Periféricos conectados a dos buses APB

Todos los dispositivos ofrecen:

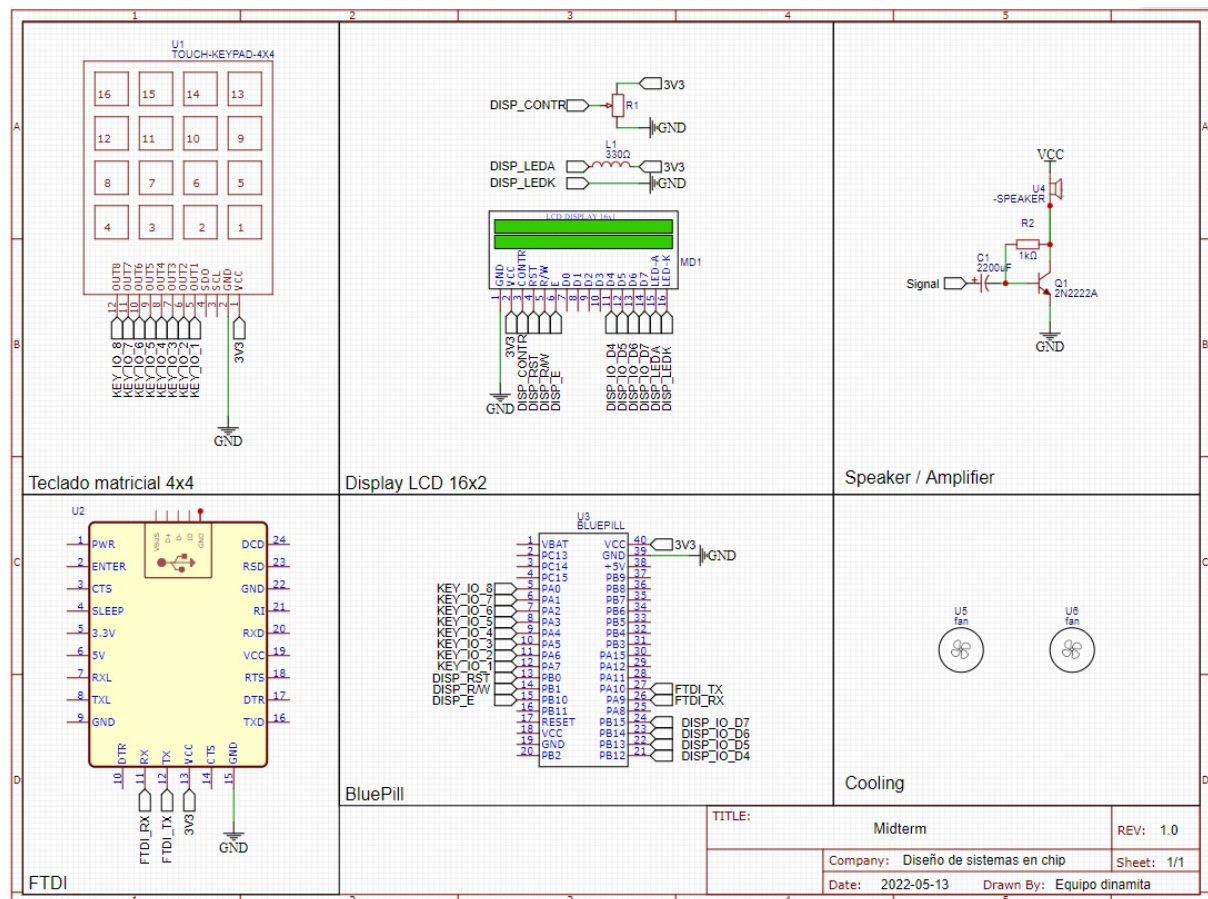
- 2 ADC de 12 bits
- 3 temporizadores de 16 bits de uso general
- 2 I2C y SPI
- 3 USART
- 1 USB

La familia de líneas de rendimiento de baja densidad STM32F103T6 funciona con una fuente de alimentación de 3.3V además de que incluye dispositivos en cuatro tipos de paquetes diferentes: de 40 pines de los cuales 26 son GPIOs.

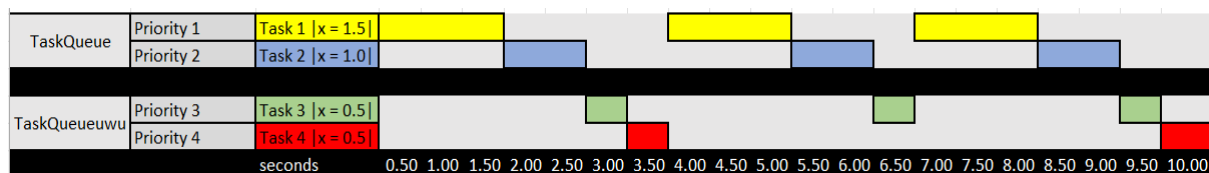
- LXTerminal

Un emulador de terminal base que está integrado en la Raspbian. Se utiliza para que el usuario pueda realizar y observar comandos en código.

Diagrama esquemático general del sistema



Línea del tiempo de calendarización de tareas del CMSIS-RTOS



- Tiempo de ejecución

Los tiempos aproximados de ejecución de cada una de las tareas son los siguientes:

• Tarea 1

- Ejecución de 1.5 segundos

- **Tarea 2**
 - Ejecución de 1 segundo
- **Tarea 3**
 - Ejecución de 0.5 segundos
- **Tarea 4**
 - Ejecución de 0.5 segundos.

Explicación de partes fundamentales del código en el STM32

Definición de Función

```

/* Private function prototypes -----*/

/* USER CODE BEGIN PFP */
/*Clock configuration and GPIO initiators*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);

/*LCD Control*/
void Usart(void);
char USER_USART1_Receive(void);

/*Keypad Control*/
void USER_RCC_Init(void);
void USER_GPIO_Init(void);
void USER_USART1_Init(void);
void senderTask1(void const * argument);
void receiverTask2(void const * argument);
void senderTask3(void const * argument);
void receiverTask4(void const * argument);

/* USER CODE END PFP */

```

En esta parte del código se definen todas las funciones a utilizar en el código del STM. Como se puede observar en cada comentario se especifica para qué parte del hardware son las funciones, a excepción de las funciones Task.

Definición de Variables

```

/* Private variables -----*/
/* USER CODE BEGIN PV */
osThreadId dTask1;//TX Keypad
osThreadId dTask2;//RX Keypad
osThreadId dTask3;//RX LCD
osThreadId dTask4;//TX LCD
osMessageQId TaskQueue; // Queue definition.
osMessageQId TaskQueueuwu; // Queue definition.
char varTemp; //Global Assist Variable
/* USER CODE END PV */

```

Aquí se definen las variables de las tareas, las queues y una variable que utilizamos para la selección de títulos en la Tarea 4.

Definición de de las queues

```
/* USER CODE BEGIN RTOS_QUEUES */

osMessageQDef(TaskQueue, 1, char);
TaskQueue = osMessageCreate(osMessageQ(TaskQueue), NULL);

osMessageQDef(TaskQueueuwu, 1, char);
TaskQueueuwu = osMessageCreate(osMessageQ(TaskQueueuwu), NULL);

/* USER CODE END RTOS_QUEUES */
```

Se hace la definición de las dos queue de datos en donde se van a guardar los datos de las tareas.

Definición y creación de Threads

```
/* Create the thread(s) */
/* definition and creation of tasks */

osThreadDef(dTask1, senderTask1, osPriorityAboveNormal, 0, 128); //osThreadDef FOR Task1
dTask1 = osThreadCreate(osThread(dTask1), NULL); //Thread creation

osThreadDef(dTask2, receiverTask2, osPriorityNormal, 0, 128); //osThreadDef FOR Task2
dTask2 = osThreadCreate(osThread(dTask2), NULL); //Thread creation

osThreadDef(dTask3, senderTask3, osPriorityBelowNormal, 0, 128); //osThreadDef FOR Task3
dTask3 = osThreadCreate(osThread(dTask3), NULL); //Thread creation

osThreadDef(dTask4, receiverTask4, osPriorityLow, 0, 128); //osThreadDef FOR Task4
dTask4 = osThreadCreate(osThread(dTask4), NULL); //Thread creation
```

While de main

```
/* USER CODE BEGIN WHILE */
while(1)
{
    senderTask1(dTask1);
    receiverTask2(dTask2);
    senderTask3(dTask3);
    receiverTask4(dTask4);
}
/* USER CODE END WHILE */
```


Función para definir los pines

```
void USER_GPIO_Init(void){
1  GPIOA->BSRR = GPIO_BSRR_BS0;
2  GPIOA->CRL &= ~GPIO_CRL_CNF0_0 & ~GPIO_CRL_MODE0;
3  GPIOA->CRL |= GPIO_CRL_CNF0_1;
4
5  GPIOA->BSRR = GPIO_BSRR_BS1;
6  GPIOA->CRL &= ~GPIO_CRL_CNF1_0 & ~GPIO_CRL_MODE1;
7  GPIOA->CRL |= GPIO_CRL_CNF1_1;
8
9  GPIOA->BSRR = GPIO_BSRR_BS2;
10 GPIOA->CRL &= ~GPIO_CRL_CNF2_0 & ~GPIO_CRL_MODE2;
11 GPIOA->CRL |= GPIO_CRL_CNF2_1;
12
13 GPIOA->BSRR = GPIO_BSRR_BS3;
14 GPIOA->CRL &= ~GPIO_CRL_CNF3_0 & ~GPIO_CRL_MODE3;
15 GPIOA->CRL |= GPIO_CRL_CNF3_1;
16
17 GPIOA->BSRR = GPIO_BSRR_BS4;
18 GPIOA->CRL &= ~GPIO_CRL_CNF4_0 & ~GPIO_CRL_MODE4_1;
19 GPIOA->CRL |= GPIO_CRL_MODE4_0;
20
21 GPIOA->BSRR = GPIO_BSRR_BS5;
22 GPIOA->CRL &= ~GPIO_CRL_CNF5_0 & ~GPIO_CRL_MODE5_1;
23 GPIOA->CRL |= GPIO_CRL_MODE5_0;
24
25 GPIOA->BSRR = GPIO_BSRR_BS6;
26 GPIOA->CRL &= ~GPIO_CRL_CNF6_0 & ~GPIO_CRL_MODE6_1;
27 GPIOA->CRL |= GPIO_CRL_MODE6_0;
28
29 GPIOA->BSRR = GPIO_BSRR_BS7;
30 GPIOA->CRL &= ~GPIO_CRL_CNF7_0 & ~GPIO_CRL_MODE7_1;
31 GPIOA->CRL |= GPIO_CRL_MODE7_0;
32
33 GPIOA->CRH &= ~GPIO_CRH_CNF9_0 & ~GPIO_CRH_MODE9_1;
34 GPIOA->CRH |= GPIO_CRH_CNF9_1 | GPIO_CRH_MODE9_0;
35
36 }
```

En este código se inicializa la configuración de los pines del teclado matricial.

Función de comunicación serial

```
288 char USER_USART1_Receive(void){
289     while ((USART1->SR & USART_SR_RXNE)==0){
290         return ',';
291     }
292     return USART1->DR;
293 }
294
```

Esta función se encarga de recibir la información de python para la interpretación de datos en las tareas 3 y 4, dedicadas al display de información de cada canción.

Función de la Task1

```
313 void senderTask1(void const * argument)
314 {
315     osStatus dTask1;
316     /* Infinite loop */
317     int a;
318
319     for(;;){
320
321         //FILA 1
322         a = 0;
323         GPIOA->ODR &= ~GPIO_ODR_ODR4; //ESTO ES 1
324         GPIOA->ODR |= GPIO_ODR_ODR5;
325         GPIOA->ODR |= GPIO_ODR_ODR6;
326         GPIOA->ODR |= GPIO_ODR_ODR7;
327         if (FILA1 == 0){
328             HAL_Delay(50);
329             a = 1;
330         }
331
332         GPIOA->ODR |= GPIO_ODR_ODR4; //ESTO ES 2
333         GPIOA->ODR &= ~GPIO_ODR_ODR5;
334         GPIOA->ODR |= GPIO_ODR_ODR6;
335         GPIOA->ODR |= GPIO_ODR_ODR7;
336         if (FILA1 == 0){
337             HAL_Delay(50);
338             a = 2;
339         }
340
341         GPIOA->ODR |= GPIO_ODR_ODR4; //ESTO ES 3
342         GPIOA->ODR |= GPIO_ODR_ODR5;
343         GPIOA->ODR &= ~GPIO_ODR_ODR6;
344         GPIOA->ODR |= GPIO_ODR_ODR7;
345         if (FILA1 == 0){
346             HAL_Delay(50);
347             a = 3;
348         }
349
350         ...
351
352         //FINAL FILA 3
353         if(a!=0){
354             dTask1 = osMessagePut(TaskQueue, a, 1000);
355             osDelay(500);
356         }
357     }
358     osDelay(5);
359 }
```

Esta función recibe el input del teclado matricial y envía la información al primer queue para que la Task2 la interprete.

Función de Task2

```
481 void receiverTask2(void const * argument)
482 {
483     osEvent dTask2;
484     /* Infinite loop */
485     for(;;){
486         dTask2 = osMessageGet(TaskQueue, 1000);
487         if( dTask2.status == osEventMessage ){
488             if(dTask2.value.v == 16)
489                 myprintf("0");
490             else if(dTask2.value.v == 1)
491                 myprintf("1");
492             else if(dTask2.value.v == 2)
493                 myprintf("2");
494             else if(dTask2.value.v == 3)
495                 myprintf("3");
496             else if(dTask2.value.v == 4)
497                 myprintf("4");
498             else if(dTask2.value.v == 5)
499                 myprintf("5");
500             else if(dTask2.value.v == 6)
501                 myprintf("6");
502             else if(dTask2.value.v == 7)
503                 myprintf("7");
504             else if(dTask2.value.v == 8)
505                 myprintf("8");
506             else if(dTask2.value.v == 9)
507                 myprintf("9");
508             else if(dTask2.value.v == 10)
509                 myprintf("#");
510             else if(dTask2.value.v == 11)
511                 myprintf("*");
512             else if(dTask2.value.v == 12)
513                 myprintf("A");
514             else if(dTask2.value.v == 13)
515                 myprintf("B");
516             else if(dTask2.value.v == 14)
517                 myprintf("C");
518             else if(dTask2.value.v == 15)
519                 myprintf("D");
520             //dTask2 = osMessagePut(TaskQueue, "1", 1000);
521         }
522     }
523 }
```

Esta función interpreta la información recibida del Task1 (El input del teclado matricial) y compara el resultado para decidir qué valor enviarle a Python. Se puede considerar el RX de la comunicación serial.

Función del Task3

```
/* USER CODE BEGIN 5 */
void senderTask3(void const * argument){
    osStatus dTask3;
    for(;;){
        varTemp = USER_USART1_Receive();
        if(varTemp != '0'){
            dTask3 = osMessagePut(TaskQueue, varTemp, 500);
            //osDelay(500);
        }
    }
}
/* USER CODE END 5 */
```

Esta función envía a la Task4 la información que fue recibida por parte de Python tras interpretar el input del teclado matricial. Se podría considerar como el TX de la comunicación serial.

Función del Task4

```
577 void receiverTask4(void const * argument){
578     osEvent dTask4;
579     for(;;){
580         dTask4 = osMessageGet(TaskQueueuuu, 500);
581         if(dTask4.value.v=='0'){
582             LCD_Clear( );//Borra la pantalla
583             LCD_Set_Cursor( 1, 0 );//Posición cursor [ 1 , 0 ].
584             LCD_Put_Str("Just like Heaven -The Cure");
585             /*LCD_Set_Cursor( 2, 0 );//Posición cursor [ 2 , 0 ].
586             LCD_Put_Str("The Cure");*/
587         } else if(dTask4.value.v=='1'){
588             LCD_Clear( );//Borra la pantalla
589             LCD_Set_Cursor( 1, 0 );//Posición cursor [ 1 , 0 ].
590             LCD_Put_Str("La Dama y el");
591             LCD_Set_Cursor( 2, 0 );//Posición cursor [ 2 , 0 ].
592             LCD_Put_Str("Moribundo - Jose Madero");
593         } else if(dTask4.value.v=='2'){
594             LCD_Clear( );//Borra la pantalla
595             LCD_Set_Cursor( 1, 0 );//Posición cursor [ 1 , 0 ].
596             LCD_Put_Str("This Charming Man");
597             LCD_Set_Cursor( 2, 0 );//Posición cursor [ 2 , 0 ].
598             LCD_Put_Str("The Smiths");
599         } else if(dTask4.value.v=='3'){
600             LCD_Clear( );//Borra la pantalla
601             LCD_Set_Cursor( 1, 0 ); //Posición cursor [ 1 , 0 ].
602             LCD_Put_Str("All Those Friendly People");
603             LCD_Set_Cursor( 2, 0 ); //Posición cursor [ 2 , 0 ].
604             LCD_Put_Str("Funeral Suits");
605         } else if(dTask4.value.v=='4'){
606             LCD_Clear( );//Borra la pantalla
607             LCD_Set_Cursor( 1, 0 );//Posición cursor [ 1 , 0 ].
608             LCD_Put_Str("Time to Pretend");
609             LCD_Set_Cursor( 2, 0 );//Posición cursor [ 2 , 0 ].
610             LCD_Put_Str("MGMT");
611         } else if(dTask4.value.v=='5'){
612             LCD_Clear( );//Borra la pantalla
613             LCD_Set_Cursor( 1, 0 );//Posición cursor [ 1 , 0 ].
614             LCD_Put_Str("Everlong");
615             LCD_Set_Cursor( 2, 0 );//Posición cursor [ 2 , 0 ].
616             LCD_Put_Str("FooFighters");
```

Esta función obtiene el valor enviado por la Task3, que se encontraba en la segunda Queue, y compara el valor para decidir qué título y artista se van a mostrar en el display.

Explicación de partes fundamentales del código en la Raspberry Pi

Comandos iniciales

```
1 # Reproduce audio con pygame
2 import pygame, serial, os, time, ctypes
3 path = "/home/piBBC/Music/Canciones/"
4 sound_files = ["Cure.wav", "Jose.wav", "Smiths.wav"]
5 cornPops = serial.Serial("/dev/ttyS0", baudrate=9600, timeout=1)
6
```

Se hace una importación de la librería “pygame”, utilizada para correr los comandos básicos de carga, reproducción y pausa de las canciones que se almacenan en el sistema. También se define la comunicación serial, estableciendo una comunicación con los demás dispositivos.

Iniciación de Pygame

```
0
7 #Iniciacion Pygame
8 pygame.mixer.init()
9 speaker_volume = 1 #100% volumen
10 pygame.mixer.music.set_volume(speaker_volume)
11
```

Se inicializa el pygame, se define el volumen de la bocina como 1 (Máximo volumen).

Clase Pausa

```
11
12 class Pause(object):
13     def __init__(self):
14         self.paused=pygame.mixer.music.get_busy()
15
16     def toggle(self):
17         if self.paused:
18             pygame.mixer.music.unpause()
19         if not self.paused:
20             pygame.mixer.music.pause()
21         self.paused = not self.paused
22
```

Definición de la clase pausa, encargado de hacer esa misma función. Se realiza la inicialización y el toggle de la pausa.

Clase Player

```
23 #class player
24 class Player:
25
26     songs = [
27         ("Cure.ogg","Just like heaven -The Cure"),
28         #("cherry.wav","Cherry Blossoms -Marselo Storm"),
29         ("Jose-converted.ogg","La Dama y El Moribundo -José Madero"),
30         #("sat.wav","Saturn's first ring -Marselo Storm"),
31         ("Smiths-converted.ogg","This Charming Man -The Smiths"),
32         ("Funeral-converted.ogg","All Those Friendly People -Funeral Suits"),
33         ("TimeTo-converted.ogg","Time To Pretend -MGMT"),
34         ("Everlong-converted.ogg","Everlong -Foo Fighters"),
35         ("EyeOf-converted.ogg","Eye Of The Tiger -Survivor"),
36         ("Dust-converted.ogg","Another One Bites The Dust -Queen"),
37         ("Woman-converted.ogg","American Woman -The Guess Who"),
38         ("RocketMan-converted.ogg","RocketMan -Elton John")
39     ]
40
41     def __init__(self):
42         self.current:int = None
43         self.previous:int = None
44
```

En la clase player se llaman a las canciones en una tupla como .ogg (una versión más versátil para código que utilizar el formato default .wav o .mp3). Además se definen los estados de la canción actual y la canción previa.

```

45 def play(self, track:int) -> None:
46     #self.current = track
47     pygame.mixer.music.load("/home/piBBC/Music/Canciones/"+self.songs[track][0])
48     pygame.mixer.music.play()
49     print(self.current)
50     print(self.previous)
51
52 def stop(self) -> None:
53     PAUSE = Pause()
54     PAUSE.toggle()
55     ...
56 def increaseVol(self) -> None:
57     global speaker_volume
58     if speaker_volume < 1:
59         speaker_volume += 0.1
60         pygame.mixer.music.set_volume(speaker_volume)
61
62 def reduceVol(self) -> None:
63     global speaker_volume
64     if speaker_volume > 0:
65         speaker_volume -= 0.1
66         pygame.mixer.music.set_volume(speaker_volume)
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108

```

Aquí se definen las funciones de play , stop, incrementar volumen y reducir volumen.

Función update

```

68
69 def update(self, processedInput) -> None:
70
71     #print(f"antes {self.current}")
72     if processedInput in ['0','1','2','3','4','5','6','7','8','9']:
73         self.stop()
74         self.previous = self.current
75         self.current = int(processedInput)
76         self.play(int(processedInput))
77     elif processedInput == '#':
78         self.increaseVol()
79     elif processedInput == '*':
80         self.reduceVol()
81     elif processedInput == 'A':
82         pygame.mixer.music.pause()
83     elif processedInput == '-4':
84         pygame.mixer.music.unpause()
85     elif processedInput == 'B':
86         pygame.mixer.music.unpause()
87
88     elif processedInput == 'C':
89         # next track
90         #print("ajhgfjghj")
91         if self.current == len(self.songs)-1:
92             self.play(0)
93             self.previous = self.current
94             self.current = 0
95         else:
96             self.play(self.current+1)
97             self.previous = self.current
98             self.current += 1
99     elif processedInput == 'D':
100         # previous track
101         if self.current == 0:
102             self.play(len(self.songs)-1)
103             self.previous = self.current
104             self.current = len(self.songs)-1
105         else:
106             self.play(self.current-1)
107             self.previous = self.current
108             self.current -= 1
109     #print(f"despues {self.current}")

```

Se define la función update, que actualiza la canción que se está reproduciendo en el momento.

Procesamiento del dato Python

```
110 def processInput(serialInput):
111     ...
112
113     serialInput:... = ...
114
115 #end class player
116
117 player=Player()
118 z=1
119 while z == 1:
120     upd = cornPops.read(1).decode("ascii")
121     player.update(upd)
122     cornPops.write(str(player.current).encode())
123
124     #print("Dato Basura")
125
```

Se define el código que interpreta el input del teclado matricial y además regresa el dato obtenido.

Conclusión

Para terminar, el proyecto nos retó a crear un sistema que se pudiera comunicar al igual que funciona un sistema de infoentretenimiento usuario-servidor. Creemos que el proyecto que desarrollamos es un proyecto que realmente demuestra nuestro aprendizaje y trabajo. Algunas problemáticas que encontramos fue que el sistema debía ser un sistema simple por los componentes que se están utilizando, al querer agregar cosas diferentes a las sugeridas se dificulta el flujo de corriente por el circuito.

Finalmente, la codificación del sistema fue la parte que al dividirlo por tareas se nos facilitó trabajarlo para poder visualizar cómo es que funcionaba la transmisión y recepción de datos enviados por el usuario. Creemos que el proyecto podía haber funcionado de una mejor manera tanto con una Raspberry pi de más rendimiento y un microcontrolador más eficiente.

Reflexiones individuales

Eduardo: Durante el proyecto logré comprender todo lo que se vio durante las 10 semanas. Creo que este reto se enfoca en la transmisión de y recepción de datos desde el STM32 a la Raspberry Pi por medio de la comunicación serial. Creo que lo que más me reto fue entender la inicialización de los GPIOs de la STM32 para que funcionaran y todos los modos a los que se pueden configurar en el CubeID. Igualmente tuvimos que entender a base de error no modificar el archivo IOC fuera del reloj, y configuraciones básicas. Finalmente, el trabajar con Python y con códigos bash en linux fue una experiencia de 100% aprendizaje para mi pues no había trabajado sobre linux anteriormente.

Luis: Este proyecto me ayudó a entender todos los temas vistos en clase. Al realizar y desarrollar este reto comprendí el funcionamiento de sistemas embebidos al trabajar con la Raspberry Pi, además de que aprendí sobre la comunicación serial entre dispositivos al tener que trasladar datos de un teclado matricial a la RPi, y viceversa. Considero que al concretar y finalizar el proyecto comprendo cómo funciona un sistema compuesto por un microcontrolador y una computadora, desarrollando mi

entendimiento sobre la funcionalidad de sistemas. Disfrute el proceso de desarrollar el código y compilar las diferentes características que fueron necesarias para hacer un sistema de infoentretenimiento, desarrollando el código en el CubeID de STM32. También refresque un poco de los temas aprendidos en otros bloques al desarrollar en Python.

Marcelo: En el desarrollo de este proyecto pudimos como equipo integrar completamente el conocimiento adquirido durante la materia, así como fortalecerlo e implementarlo de múltiples maneras aplicando nuestra creatividad. Por si fuera poco, decidimos complementar el prototipo con cualidades que lo vuelven único y mejoran su funcionalidad en diversos aspectos.

Octavio: El trabajar en este proyecto para mí fue un salto de los proyectos que veníamos estado haciendo a proyectos más profesionales y complejos que ya usan conceptos más conceptos y esos conceptos llegan a ser más abstractos por lo que nos dan más libertad para trabajar. Juntamos todo el trabajo de distintos semestres y aprendimos nuevos. Lo más importante fue el trabajar con RTOS ya que hicimos un pequeño prototipo de uno de estos en los que trabajamos con tiempos, con tasks, con queues, aprendimos a configurar pines para distintos propósitos y a establecer conexiones con distintos componentes para poder optimizar tareas y trabajar con distintas plataformas lo que nos abre una infinidad de posibilidades y el poder concretar el proyecto y darle un cierre nos dio una perspectiva distinta de lo que significa estudiar robótica y me dio un aprendizaje íntegro de todo el software y hardware del proyecto.

Referencias

- Redhat(2019) Linux y bash. Recuperado de: <https://www.redhat.com/es/topics/linux> el 14 de Junio del 2022
- Components(2021) Raspberry 3 Pinouts, Features and more. Recuperado de: <https://components101.com/microcontrollers/raspberry-pi-3-pinout-features-datasheet> el 14 Junio del 2022
- Programmer Click (2022)STM y configuraciones. Recuperado de: <https://programmerclick.com/article/56612100804/> el 14 de Junio